

Analytical Problem Solving

Name:Manoj M(192511060)

Question 1: Banking Account Management System

1. Define the Account class with suitable data members.

The Account class contains the following data members:

- accountNumber – stores the account number.
- customerName – stores the customer name.
- balance – stores the account balance.

2. Implement the default, parameterized, and copy constructors.

3. Create objects using each constructor.

4. Display the details of each account.

```
#include <iostream>
using namespace std;

class Account
{ private:
    int accountNumber;
    string customerName;
    double balance;

public:
    // Default Constructor
    Account()
    {
        accountNumber = 0;
        customerName = "Unknown";
        balance = 0.0;

        cout << "Default Constructor Called" << endl;
    }

    // Parameterized Constructor
    Account(int accNo, string name, double bal)
    {
        accountNumber = accNo;
        customerName = name;
        balance = bal;

        cout << "Parameterized Constructor Called" << endl;
    }

    // Copy Constructor
    Account(const Account &a)
    {
```

```

        accountNumber = a.accountNumber;
customerName = a.customerName;    balance
= a.balance;

        cout << "Copy Constructor Called" << endl;
    }

    // Display Function
    void display()
    {
        cout << "Account Number : " << accountNumber << endl;
cout << "Customer Name : " << customerName << endl;
cout << "Balance      : " << balance << endl;    cout <<
endl;
    }
};

int main()
{
    // Object using Default Constructor
    Account a1;
    a1.display();

    // Object using Parameterized Constructor
    Account a2(101, "Ragul", 25000);
    a2.display();

    // Object using Copy Constructor
    Account a3(a2);    a3.display();

    return 0; }

```

Output

```

Default Constructor Called
Account Number : 0
Customer Name : Unknown
Balance      : 0

```

```

Parameterized Constructor Called
Account Number : 101
Customer Name : Ragul
Balance      : 25000

```

```

Copy Constructor Called
Account Number : 101
Customer Name : Ragul
Balance      : 25000

```

SCREENSHORT:

```
main.cpp
1 #include <iostream>
2 using namespace std;
3
4 class Account
5 {
6 private:
7     int accountNumber;
8     string customerName;
9     double balance;
10
11 public:
12
13     Account()
14     {
15         accountNumber = 0;
16         customerName = "Unknown";
17         balance = 0.0;
18
19         cout << "Default Constructor Called" << endl;
20     }
21
22
23     Account(int accNo, string name, double bal)
24     {
25         accountNumber = accNo;
26         customerName = name;
27         balance = bal;
28
29
30     }
31 }
```

input

```
Account Number : 101
Customer Name : Ragul
Balance : 25000
```

5. Analyze and explain how object initialization differs for the three constructors.

Constructor	Purpose	Example
Default Constructor	Initializes an object with default values	Account a1;
Parameterized Constructor	Initializes an object with given values	Account a2(101, "Ragul", 25000);
Copy Constructor	Creates a new object by copying an existing object	Account a3(a2);

Default Constructor:

It does not require any arguments and assigns default values to the data members.

Parameterized Constructor:

It accepts arguments and initializes the account with the values provided by the user.

Copy Constructor:

It accepts an existing object as an argument and copies its data into a new object.

6. Create a new account object from an existing account object using the copy constructor.

Account a3(a2);

Here, a3 is created by copying the details of a2.

a2 → Account Number = 101
Customer Name = Ragul
Balance = 25000

a3 → Account Number = 101
Customer Name = Ragul
Balance = 25000

7. Predict and explain the order in which constructors are called during object creation and copying.

The objects are created in this order:

Account a1;
Account a2(101, "Ragul", 25000); Account
a3(a2);

Therefore, the constructor calling order is:

1. Default Constructor → a1
2. Parameterized Constructor → a2
3. Copy Constructor → a3

Question 2: Inventory Product Management System

1. Design the `Product` class with appropriate data members.

The `Product` class contains the following data members:

- `productId` – stores the product ID.
- `productName` – stores the product name.
- `stock` – stores the stock quantity.

2. Implement a parameterized constructor to initialize product details.

The parameterized constructor accepts product ID, product name, and stock as arguments and initializes the data members.

3. Implement a copy constructor to duplicate a product.

The copy constructor creates a new product by copying the details of an existing product.

4. Implement a destructor to release resources when an object is destroyed.

The destructor is automatically called when an object is destroyed. It is used to release resources if required.

5. Overload the `+` operator to add the stock quantities of two products.

The `+` operator is overloaded to add the stock quantities of two product objects.

`20 + 30 = 50`

6. Create two product objects using the parameterized constructor.

```
Product p1(101, "Laptop", 20);  
Product p2(102, "Mouse", 30);
```

7. Create another product using the copy constructor.

```
Product p3(p1);
```

Here, p3 is created by copying the details of p1.

8. Add two products using the overloaded + operator.

Product p4 = p1 + p2;

The stock quantities are added:

20 + 30 = 50

9. Display the product details and resulting stock quantity.

The display() function displays the product ID, product name, and stock quantity.

10. Trace and explain the sequence of constructor and destructor calls.

Constructor order:

1. Parameterized Constructor → p1
2. Parameterized Constructor → p2
3. Copy Constructor → p3
4. Parameterized Constructor → p4

Destructor order:

Objects are destroyed in reverse order:

1. Destructor → p4
2. Destructor → p3
3. Destructor → p2
4. Destructor → p1

Complete C++ Program

```
#include <iostream>
using namespace std;
```

```
class Product
{ private:
    int productId;
    string productName;
    int stock;
```

```

public:
    // Parameterized Constructor
    Product(int id, string name, int s)
    {
        productId = id;
        productName = name;
        stock = s;

        cout << "Parameterized Constructor Called" << endl;
    }

    // Copy Constructor
    Product(const Product &p)
    {
        productId = p.productId;
        productName = p.productName;
        stock = p.stock;

        cout << "Copy Constructor Called" << endl;
    }

    // Overloaded + Operator
    Product operator+(const Product &p)
    {
        Product
        result(
            productId,
            productName + " + " + p.productName,
            stock + p.stock
        );

        return result;
    }

    // Display Function
    void display()
    {
        cout << "Product ID : " << productId << endl;
        cout << "Product Name : " << productName << endl;
        cout << "Stock : " << stock << endl;
        cout << endl;
    }

    // Destructor
    ~Product()
    {
        cout << "Destructor Called for "
              << productName << endl;
    }
};

int main()
{
    // 6. Create two products
    Product p1(101, "Laptop", 20);
    Product p2(102, "Mouse", 30);

```



```

// 7. Create product using copy constructor
Product p3(p1);

// 8. Add two products
Product p4 = p1 + p2;

// 9. Display details    cout <<
"\nProduct 1 Details:" << endl;
p1.display();

cout << "Product 2 Details:" << endl;
p2.display();

cout << "Copied Product Details:" << endl;
p3.display();

cout << "Combined Product Details:" << endl;
p4.display();

return 0; }

```

Output

Parameterized Constructor Called
 Parameterized Constructor Called
 Copy Constructor Called
 Parameterized Constructor Called

Product 1 Details: Product
 ID : 101
 Product Name : Laptop
 Stock : 20

Product 2 Details: Product
 ID : 102
 Product Name : Mouse
 Stock : 30

Copied Product Details:
 Product ID : 101
 Product Name : Laptop
 Stock : 20

Combined Product Details:
 Product ID : 101
 Product Name : Laptop + Mouse
 Stock : 50

Destructor Called for Laptop + Mouse
 Destructor Called for Laptop
 Destructor Called for Mouse
 Destructor Called for Laptop

SCREENSHOT:

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  class Product
5  {
6  private:
7      int productId;
8      string productName;
9      int stock;
10
11 public:
12     // Parameterized Constructor
13     Product(int id, string name, int s)
14     {
15         productId = id;
16         productName = name;
17         stock = s;
18
19         cout << "Parameterized Constructor Called" << endl;
20     }
21
22     // Copy Constructor
23     Product(const Product &p)
24     {
25         productId = p.productId;
26         productName = p.productName;
27         stock = p.stock;
28
29     }
30
31     ~Product()
32     {
33         cout << "Destructor Called for " << productName << endl;
34     }
35 }
```

input

```
Destructor Called for Laptop + Mouse
Destructor Called for Laptop
```

Question 3: Shopping Cart Management System

1. Design the `Product` class with appropriate data members.

The `Product` class contains:

- `productName` –

stores the product name.

- `quantity` – stores the quantity of the product.

2. Implement a parameterized constructor to initialize product details.

The parameterized constructor initializes the product name and quantity.

3. Implement a copy constructor to duplicate a product.

The copy constructor creates a new object by copying the details of an existing object.

4. Implement a destructor to release resources when an object is destroyed.

The destructor is automatically called when an object is destroyed.

5. Overload the + operator to add the stock quantities of two products.

The + operator adds the quantities of two products.

For example:

$$2 + 3 = 5$$

6. Create two product objects using the parameterized constructor.

```
ShoppingCart c1("Laptop", 2);  
ShoppingCart c2("Mouse", 3);
```

7. Create another product using the copy constructor.

```
ShoppingCart c3(c1);
```

Here, c3 is created by copying c1.

8. Add two products using the overloaded + operator.

```
ShoppingCart c4 = c1 + c2;
```

The quantities are added:

$$2 + 3 = 5$$

9. Display the product details and resulting stock quantity.

The display() function displays the product name and quantity.

The merged cart contains:

Product Name : Laptop + Mouse

Quantity : 5

10. Trace and explain the sequence of constructor and destructor calls.

Constructor order

1. Parameterized Constructor → c1
2. Parameterized Constructor → c2
3. Copy Constructor → c3
4. Parameterized Constructor → c4

Destructor order

Objects are destroyed in reverse order:

1. Destructor → c4
2. Destructor → c3
3. Destructor → c2
4. Destructor → c1

11. Explain what may happen if dynamically allocated memory is not released in the destructor.

If dynamically allocated memory is not released, a **memory leak** can occur.

For example:

```
int *p = new int;
```

The memory should be released using:

```
delete p;
```

If it is not released:

- Memory remains occupied.
- Available memory decreases.
- The program may consume more memory.
- Long-running programs may become slower.

Therefore, the destructor should release dynamically allocated memory when required.

Complete C++ Program

```
#include <iostream>
using namespace std;

class ShoppingCart
{ private:  string
  productName;  int
  quantity;

public:
  // Parameterized Constructor
  ShoppingCart(string name, int qty)
  {
    productName = name;
    quantity = qty;

    cout << "Parameterized Constructor Called" << endl;
  }

  // Copy Constructor
  ShoppingCart(const ShoppingCart &c)
  {
    productName = c.productName;
    quantity = c.quantity;

    cout << "Copy Constructor Called" << endl;
  }

  // Overloaded + Operator
  ShoppingCart operator+(const ShoppingCart &c)
  {
    ShoppingCart
result(      productName + " + " +
c.productName,
    quantity + c.quantity
    );

    return result;
  }

  // Display Function
  void display()
  {
    cout << "Product Name : " << productName << endl;
cout << "Quantity    : " << quantity << endl;
    cout << endl;
  }

  // Destructor
  ~ShoppingCart()
  {
    cout << "Destructor Called for "
    << productName << endl;
  }
}
```

```

    }
};

int main()
{
    // 6. Create two product objects
    ShoppingCart c1("Laptop", 2);
    ShoppingCart c2("Mouse", 3);

    // 7. Create object using copy constructor
    ShoppingCart c3(c1);

    // 8. Add two products
    ShoppingCart c4 = c1 + c2;

    // 9. Display details    cout <<
    "\nCart 1 Details:" << endl;
    c1.display();

    cout << "Cart 2 Details:" << endl;
    c2.display();

    cout << "Copied Cart Details:" << endl;
    c3.display();

    cout << "Merged Cart Details:" << endl;
    c4.display();

    return 0;
}

```

Output

Parameterized Constructor Called
 Parameterized Constructor Called
 Copy Constructor Called
 Parameterized Constructor Called

Cart 1 Details:
 Product Name : Laptop
 Quantity : 2

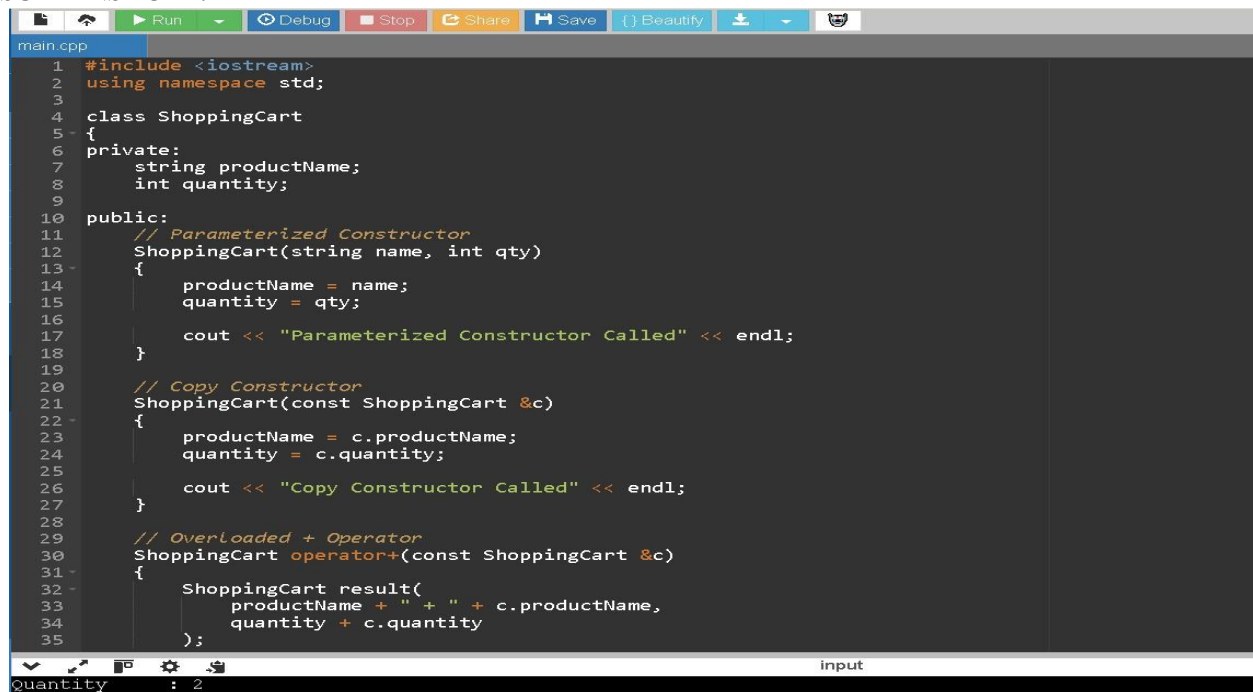
Cart 2 Details:
 Product Name : Mouse
 Quantity : 3

Copied Cart Details:
 Product Name : Laptop
 Quantity : 2

Merged Cart Details:
 Product Name : Laptop + Mouse
 Quantity : 5

Destructor Called for Laptop + Mouse
Destructor Called for Laptop
Destructor Called for Mouse
Destructor Called for Laptop

SCREENSHOT:



```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  class ShoppingCart
5  {
6  private:
7      string productName;
8      int quantity;
9
10 public:
11     // Parameterized Constructor
12     ShoppingCart(string name, int qty)
13     {
14         productName = name;
15         quantity = qty;
16         cout << "Parameterized Constructor Called" << endl;
17     }
18
19     // Copy Constructor
20     ShoppingCart(const ShoppingCart &c)
21     {
22         productName = c.productName;
23         quantity = c.quantity;
24         cout << "Copy Constructor Called" << endl;
25     }
26
27     // Overloaded + Operator
28     ShoppingCart operator+(const ShoppingCart &c)
29     {
30         ShoppingCart result(
31             productName + " + " + c.productName,
32             quantity + c.quantity
33         );
34     }
35 }
```

quantity : 2